

Ticket Management System

Documentazione di Progetto — Infrastruttura Azure & Applicazione

Progetto di pratica DevOps · Terraform + Azure + Container Apps
Preparato in preparazione alla certificazione AZ-305 (Azure Solutions Architect)
Agosto 2026

Indice

1. Panoramica architetturale

2. Organizzazione Terraform e isolamento degli ambienti

3. Componenti infrastrutturali

3.1 Storage Account — state Terraform

3.2 Resource Group

3.3 Azure Container Registry

3.4 Log Analytics Workspace

3.5 Key Vault

3.6 Azure SQL Database (Server + Database)

3.7 Container Apps Environment

3.8 Managed Identity (User-Assigned)

3.9 Container App — ticket-api

3.10 Container App — ticket-web

4. Rete — vista d'insieme

5. IAM — vista d'insieme

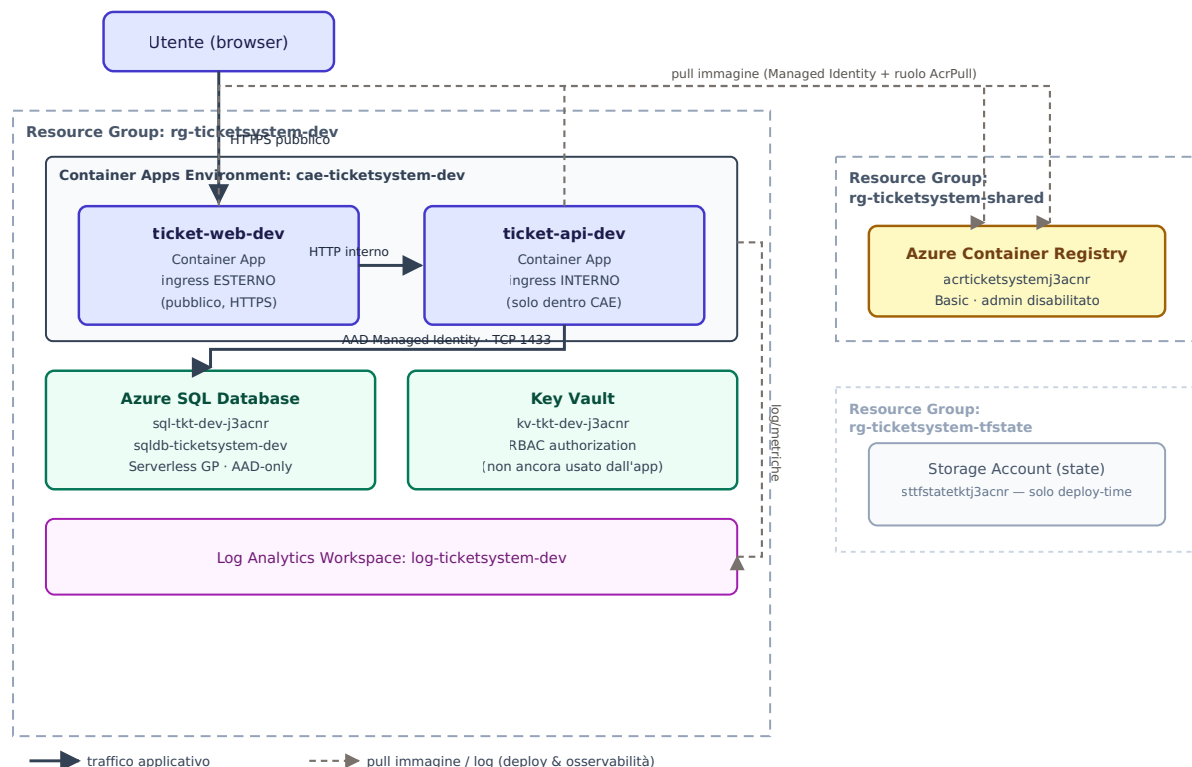
6. Applicazione — ticket-api

7. Applicazione — ticket-web

8. Stato del progetto e prossimi passi

1. Panoramica architetturale

Il sistema è composto da due servizi applicativi indipendenti (`ticket-api` e `ticket-web`), entrambi in .NET 8, che girano come Container App dentro un unico Container Apps Environment in Consumption plan. `ticket-web` è l'unica componente esposta pubblicamente; `ticket-api` è raggiungibile solo dall'interno dell'ambiente. I dati sono su Azure SQL Database (Serverless General Purpose), le immagini Docker vivono in un Azure Container Registry condiviso tra gli ambienti, e ogni identità di calcolo si autentica tramite Managed Identity — nessuna password è salvata da nessuna parte del sistema.



Il Resource Group `rg-ticketsystem-tfstate` (in basso a destra nel diagramma) non fa parte del flusso di traffico applicativo: contiene solo lo storage account dove Terraform salva il proprio state, usato esclusivamente al momento del deploy (`terraform apply`), mai a runtime dall'applicazione.

Flusso di una richiesta utente

1. Il browser apre l'URL pubblico di `ticket-web` (HTTPS, certificato gestito da Container Apps).
2. Blazor Server mantiene una connessione SignalR persistente col browser: tutta la UI viene renderizzata lato server.
3. Quando l'utente crea/modifica/elimina un ticket, `ticket-web` (lato server) chiama `ticket-api` via HTTP, usando l'FQDN **interno** di quest'ultimo — raggiungibile solo da dentro lo stesso Container Apps Environment.

4. `ticket-api` esegue la query su Azure SQL, autenticandosi con la propria Managed Identity (nessuna password in nessun file di configurazione).
5. La risposta risale la stessa catena fino al browser.

2. Organizzazione Terraform e isolamento degli ambienti

L'infrastruttura è definita interamente in Terraform, organizzata secondo il pattern **moduli riutilizzabili + root configuration per ambiente** — lo stesso raccomandato dalla documentazione ufficiale HashiCorp per progetti che richiedono isolamento reale tra ambienti (non solo logico).

Cartella	Ruolo
<code>infra/modules/*</code>	Componenti riutilizzabili e parametrizzati (nessun <code>provider</code> / <code>backend</code> proprio). Richiamati dagli ambienti.
<code>infra/environments/shared</code>	State indipendente. Risorse condivise tra tutti gli ambienti: Resource Group + Azure Container Registry.
<code>infra/environments/dev</code>	State indipendente. Tutte le risorse dell'ambiente di sviluppo/test.
<code>infra/environments/staging, prod</code>	Predisposti (solo <code>backend.tf</code>), risorse non ancora definite.

Perché uno state separato per "shared": il pattern applicativo è "build once, promote many" — un'unica immagine Docker viene taggata e promossa tra `dev` → `staging` → `prod`, non ricostruita per ognuno. Se l'ACR visse nello state di `dev`, un `terraform destroy` su `dev` (frequente in questo progetto per contenere i costi) distruggerebbe anche il registry usato da `staging/prod`. Con state separati, `destroy` su `dev` non ha letteralmente nessuna conoscenza di cosa esiste nello state di `shared`.

L'ambiente `dev` legge in sola lettura un output dello state di `shared` (l'hostname dell'ACR) tramite un data source `terraform_remote_state`, puntato allo stesso storage account/container usato dal proprio `backend.tf`, ma con `key = "shared.terraform.tfstate"`.

Sicurezza dello state

- Storage account con `allow_shared_key_access = false`: unica autenticazione possibile è Azure AD/RBAC, nessuna Storage Account Key.
- Versioning + soft delete (7gg) sul blob container, per recovery in caso di state corrotto.
- RBAC assegnato in scope sul singolo storage account (ruolo Storage Blob Data Contributor), non sulla subscription.

3. Componenti infrastrutturali

Per ogni componente: cosa è, perché è stato scelto (incluse le alternative valutate), come funziona la rete, come funziona l'accesso (IAM), e la rilevanza per l'esame AZ-305.

3.1 Storage Account — state Terraform

Risorsa: `sttfstatetktj3acnr` (Standard_LRS, StorageV2), creato manualmente una tantum dallo script `Bootstrap/00-bootstrap-tfstate.ps1` — non da Terraform stesso (problema "chicken-and-egg": Terraform ha bisogno di un backend prima di poter gestire alcuna risorsa, incluso il proprio backend).

Perché LRS: ridondanza locale nella sola regione `switzerlandnorth`, sufficiente per un progetto di test in singola regione. ZRS/GRS/GZRS aggiungerebbero costo per un RPO/RTO che questo progetto non richiede.

Rete: endpoint pubblico dello storage account, nessuna restrizione di rete (né firewall IP né private endpoint) — protetto esclusivamente dal livello IAM sottostante.

IAM: `allow_shared_key_access = false` — niente Storage Account Key. Unico accesso possibile: la propria identità Azure AD con ruolo Storage Blob Data Contributor assegnato in scope sul singolo storage account.

AZ-305: LRS vs ZRS vs GRS vs GZRS (RPO/RTO), autenticazione basata su identità vs shared key (Zero Trust, Well-Architected Framework — pilastro Security), versioning/soft-delete come meccanismo di recovery.

3.2 Resource Group

Risorse: `rg-ticketsystem-tfstate`, `rg-ticketsystem-shared`, `rg-ticketsystem-dev` (staging/prod predisposti, non ancora creati). Naming ispirato ad Azure CAF.

Perché un RG per ambito invece di uno unico: il Resource Group è il confine naturale per RBAC, Azure Policy e lifecycle. Ambiti diversi (state Terraform, risorse condivise, ambiente dev) hanno lifecycle indipendenti — un errore in uno non deve poter propagarsi agli altri.

AZ-305: il Resource Group come unità di scoping per RBAC/Policy/lifecycle è un concetto cardine di "Design governance".

3.3 Azure Container Registry

Risorsa: `acrticketssystemj3acnr`, SKU **Basic**, `admin_enabled = false`. Vive nell'ambiente `shared`.

Perché Basic (non Standard/Premium): throughput/storage limitati e nessuna geo-replica, sufficienti per un progetto mono-team mono-regione. Premium sbloccherebbe geo-replicazione, private endpoint e customer-managed keys — non necessari qui.

Rete: endpoint pubblico, nessuna restrizione di rete configurata (il tier Basic non supporta comunque private endpoint).

IAM: admin account disabilitato. Ogni identità che deve interagire con l'ACR riceve un ruolo RBAC puntuale: `AcrPull` assegnato alle Managed Identity di `ticket-api-dev` e `ticket-web-dev`, in scope sull'ACR stesso.

AZ-305: scelta SKU (throughput/geo-replica), pattern "build once, deploy many", disabilitazione admin account + RBAC come principio "least privilege" (stesso tema di Key Vault e Storage Account).

3.4 Log Analytics Workspace

Risorsa: `log-ticketssystem-dev`, SKU **PerGB2018**, retention 30 giorni. Un workspace per ambiente (non condiviso).

Perché PerGB2018 (pay-as-you-go): nessun impegno di capacità minima giornaliera, corretto per un progetto a basso traffico e finestre di test brevi. Una Capacity Reservation converrebbe solo con ingestion costante e prevedibile.

Rete: endpoint pubblico per query/ingestion, nessuna restrizione configurata.

IAM: nessun ruolo custom assegnato — il collegamento con il Container Apps Environment avviene a livello di configurazione risorsa (`log_analytics_workspace_id`), non di RBAC utente.

AZ-305: workspace centralizzato vs per-ambiente/team (trade-off costo query cross-workspace vs isolamento/governance dei dati), scelta SKU pricing.

3.5 Key Vault

Risorsa: `kv-tkt-dev-j3acnr` (nome abbreviato "tkt" per il limite di 24 caratteri di Azure), SKU standard. `rbac_authorization_enabled = true`, `purge_protection_enabled = false`, `soft_delete_retention_days = 7` (minimo consentito).

Perché RBAC invece di Access Policies: le Access Policies sono un modello di autorizzazione legacy, non integrato con Entra ID RBAC. Il modello RBAC è lo stesso usato per tutte le altre risorse del progetto, auditabile, coerente con Managed Identity e OIDC.

Perché purge protection disabilitata: scelta deliberata per QUESTO progetto — con purge protection attiva, un vault eliminato resta "soft-deleted" e blocca la ricreazione di un vault con lo stesso nome fino alla scadenza della retention. Per cicli rapidi di destroy/apply in un tenant di test sarebbe solo attrito. In un ambiente prod reale, purge protection andrebbe attivata.

Rete: endpoint pubblico, nessuna restrizione di rete configurata.

IAM: RBAC authorization abilitata, ma **nessun ruolo ancora assegnato** — il Key Vault è predisposto per ospitare secret futuri (nessuna applicazione lo usa ancora, dato che l'autenticazione a SQL è AAD-diretta, non tramite secret in Key Vault).

AZ-305: RBAC vs Access Policies, soft-delete/purge protection come "recovery from accidental deletion" (Well-Architected Framework — pilastro Security).

3.6 Azure SQL Database (Server + Database)

Risorse: logical server `sql-tkt-dev-j3acnr` + database `sqlldb-ticketsystem-dev`, SKU **GP_S_Gen5_1** (General Purpose, **Serverless**, 1 vCore), `min_capacity = 0.5`, `auto_pause_delay_in_minutes = 60`, `max_size_gb = 2`.

Serverless vs Provisioned: Serverless scala automaticamente i vCore e va in **auto-pause** dopo 60 minuti di inattività (compute fatturato a zero, si paga solo lo storage) — la leva di risparmio più importante del progetto insieme allo scale-to-zero dei Container Apps. Provisioned sarebbe corretto solo con carico costante e prevedibile (tipico di prod).

DTU vs vCore: scelta esplicita vCore — il modello DTU è un bundle opaco di CPU/memoria/IO che non consente scaling granulare né la modalità serverless.

Hyperscale: scartato — pensato per database molto grandi (>100GB) con scaling storage indipendente, over-engineering per un ticket system di pratica.

Rete: endpoint pubblico (nessun VNet/Private Endpoint in questo progetto — introdurrebbe una subnet dedicata e una Private DNS Zone, complessità non giustificata qui). Accesso limitato da due regole firewall:

- `AllowAzureServices` (0.0.0.0-0.0.0.0): necessaria perché il Container Apps Environment Consumption non ha VNet integration di default e raggiunge SQL dal suo endpoint pubblico.
- `AllowMyIp`: l'IP pubblico della postazione di sviluppo, per test diretti da locale (`dotnet run`, Azure Data Studio).

IAM: server **AAD-only** (`azuread_authentication_only = true`) — nessun login SQL classico esiste, niente `administrator_login/password`. L'amministratore è un'identità Entra ID (l'utente stesso). L'app `ticket-api` si connette con la propria Managed Identity; per funzionare, la Managed Identity deve prima essere creata come utente del database con uno

script T-SQL manuale (`CREATE USER [...] FROM EXTERNAL PROVIDER`) — Terraform/il provider `azurerm` non espone una risorsa per gestire utenti/permessi a livello di database SQL.

AZ-305: DTU vs vCore, Provisioned vs Serverless vs Hyperscale, Public Endpoint+firewall vs Private Endpoint — tutti temi espliciti di "Design data storage solutions"; autenticazione Entra-only/passwordless (pilastro Security del WAF).

3.7 Container Apps Environment

Risorsa: `cae-ticketsystem-dev`, Consumption plan (nessun workload profile dedicato), collegato al Log Analytics Workspace dell'ambiente.

Perché Consumption invece di workload profiles dedicati: nessuna infrastruttura riservata — si paga per vCPU-secondi/GiB-secondi effettivamente usati, con scale-to-zero nativo. Workload profiles dedicati (D4/D8/E-series) riserverebbero VM con costo fisso anche a carico zero, giustificati solo per requisiti hardware specifici (GPU, memory-optimized) o isolamento multi-tenant.

Rete: rete virtuale gestita internamente da Azure (non visibile/configurabile in modalità Consumption-only, nessuna VNet del cliente coinvolta). Espone un dominio di default (`*.<hash>.<region>.azurecontainerapps.io`) usato sia per gli endpoint pubblici sia per quelli interni (`*.internal...`).

IAM: nessuna identità propria — il collegamento a Log Analytics avviene via configurazione risorsa, non RBAC.

AZ-305: Container Apps confrontato con App Service/AKS/Functions come opzione di hosting compute — Consumption plan con scale-to-zero è l'argomento chiave per "cloud-native microservices senza gestione di cluster" a costo minimo in assenza di traffico.

3.8 Managed Identity (User-Assigned) — `id-ticket-api-dev`, `id-ticket-web-dev`

Risorse: due `azurerm_user_assigned_identity`, una per Container App, create come risorse indipendenti PRIMA delle Container App che le useranno.

Perché User-Assigned e non System-Assigned: scelta corretta durante il deploy, dopo aver diagnosticato un problema reale. Con un'identità di sistema, l'identità nasce insieme alla Container App — quindi il ruolo `AcrPull` non può ancora esistere quando Azure tenta il primo pull dell'immagine, che avviene **durante la creazione stessa della Container App** (non solo alla prima richiesta reale, contrariamente a quanto ci si aspetterebbe da uno scale-to-zero con `min_replicas = 0`). Risultato osservato: la creazione della Container App resta bloccata per minuti in un loop di tentativi falliti (401 Unauthorized), a volte fino a fallire per timeout ("Operation expired"). Con una User-Assigned Identity creata come risorsa separata e con il

ruolo già assegnato PRIMA di creare la Container App (esplicito `depends_on` in Terraform), il primo pull funziona al primo tentativo.

IAM: ruolo `AcrPull` assegnato al `principal_id` di ciascuna identità, in scope sull'ACR condiviso — assegnato PRIMA di essere referenziato da qualunque Container App.

AZ-305: User-Assigned vs System-Assigned Managed Identity — lifecycle indipendente e riutilizzabile su più risorse vs 1:1 e legato al lifecycle della risorsa che la possiede — è un confronto esplicito dell'esame.

3.9 Container App — ticket-api

Risorsa: `ticket-api-dev`. Ingress **interno** (`external_enabled = false`), porta target 8080. `min_replicas = 0`, `max_replicas = 1`, 0.25 vCPU / 0.5Gi memoria. Identity: **User-Assigned** (`id-ticket-api-dev`).

Perché ingress interno: nessun consumatore esterno del servizio — solo `ticket-web`, dentro lo stesso Container Apps Environment, lo chiama. Nessun endpoint pubblico non necessario ("minimal exposed surface", stesso principio applicato al firewall SQL e all'RBAC di Key Vault).

Perché `min_replicas = 0`: costo compute a zero quando non c'è traffico.

Rete: raggiungibile solo dall'FQDN interno (`*.internal.<default_domain>`), non risolvibile/instradabile da fuori il Container Apps Environment. In uscita: verso Azure SQL (endpoint pubblico + firewall) e verso l'ACR (pull immagine).

IAM: usa la User-Assigned Identity `id-ticket-api-dev` con due ruoli/permessi:

- `AcrPull` (RBAC Azure) sull'ACR condiviso — per scaricare la propria immagine.
- Utente database (`db_datareader` + `db_datawriter`) su `sqlldb-ticketsystem-dev`, creato via T-SQL (`CREATE USER [id-ticket-api-dev] FROM EXTERNAL PROVIDER`) — per leggere/scrivere i ticket.

Connection string: `Authentication=Active Directory Managed Identity;UserId=<client_id>` — il `client_id` identifica quale identità usare (necessario con User-Assigned, non richiesto con System-Assigned). Nessuna password nella configurazione della Container App.

AZ-305: segmentazione di rete (ingress interno vs esterno) come principio di "minimal exposure" applicato al livello PaaS; Managed Identity come pattern passwordless standard del WAF.

3.10 Container App — ticket-web

Risorsa: `ticket-web-dev`. Ingress **esterno** (`external_enabled = true`), porta target 8080. `min_replicas = 0`, `max_replicas = 1`, 0.25 vCPU / 0.5Gi memoria. Identity: **User-Assigned** (`id-ticket-web-dev`).

Perché ingress esterno: è l'unica componente che deve essere raggiungibile da un browser — la UI del sistema.

Rete: endpoint pubblico HTTPS (`*.<default_domain>`), certificato TLS gestito automaticamente da Container Apps. In uscita: verso `ticket-api` (FQDN interno) e verso l'ACR (pull immagine). Nessun accesso diretto al database.

IAM: usa la User-Assigned Identity `id-ticket-web-dev` con un solo ruolo: `AcrPull` sull'ACR condiviso. Nessun accesso a Key Vault o SQL — `ticket-web` non ha bisogno di credenziali proprie, comunica con `ticket-api` via chiamate HTTP semplici (nessuna autenticazione servizio-a-servizio configurata al momento, essendo l'ingress interno già di per sé non raggiungibile dall'esterno dell'environment).

AZ-305: stesso principio di least-privilege applicato alle identità applicative — ogni Managed Identity ha esattamente i permessi che le servono, non di più.

4. Rete — vista d'insieme

Il progetto non utilizza alcuna Virtual Network: ogni risorsa PaaS espone il proprio endpoint pubblico gestito da Azure, e l'accesso è ristretto tramite firewall (solo SQL) o IAM (tutto il resto). Questa è una scelta deliberata di semplicità per un progetto di pratica — un ambiente di produzione reale userebbe tipicamente VNet integration/Private Endpoint per SQL e Key Vault.

Componente	Esposizione	Protezione
ticket-web	Pubblica (HTTPS)	Nessuna (è l'entry point voluto del sistema)
ticket-api	Interna al Container Apps Environment	DNS/routing non risolvibile dall'esterno
Azure SQL	Pubblica (TCP 1433)	Firewall: solo Azure services + IP dello sviluppatore
Key Vault	Pubblica	Nessuna restrizione di rete — solo RBAC (nessun ruolo ancora assegnato)
Azure Container Registry	Pubblica	Nessuna restrizione di rete (Basic SKU) — solo RBAC (AcrPull)
Storage Account (state)	Pubblica	Nessuna restrizione di rete — solo RBAC + shared key disabilitata

5. IAM — vista d'insieme

Identità	Tipo	Ruolo / permesso	Scope
Utente (sviluppatore)	Account Entra ID	Storage Blob Data Contributor	Storage account state Terraform
Utente (sviluppatore)	Account Entra ID	Azure AD Administrator del server SQL (controllo completo)	SQL Server dev
id-ticket-api-dev (usata da ticket-api-dev)	Managed Identity (User-Assigned)	AcrPull	Azure Container Registry
id-ticket-api-dev (usata da ticket-api-dev)	Managed Identity (User-Assigned)	db_datareader + db_datawriter (utente DB, via T-SQL)	Database sqldb-ticketsystem-dev
id-ticket-web-dev (usata da ticket-web-dev)	Managed Identity (User-Assigned)	AcrPull	Azure Container Registry

Nessuna credenziale statica (password, chiave, connection string con secret) è presente in alcun file di configurazione o variabile d'ambiente del progetto. Ogni autenticazione compute-to-PaaS avviene tramite Managed Identity; l'autenticazione da locale (sviluppo) avviene tramite l'identità Azure CLI dello sviluppatore (`az login`).

6. Applicazione — ticket-api

ASP.NET Core Web API (.NET 8) — CRUD ticket, EF Core su Azure SQL.

File	Responsabilità
Models/Ticket.cs	Entità: Id, Title, Description, Status (enum), Priority (enum), CreatedAt, UpdatedAt.
Data/TicketDbContext.cs	DbContext EF Core. Enum salvati come stringa nel DB (leggibilità in strumenti di ispezione diretta).
Controllers/TicketsController.cs	Endpoint REST sotto /api/tickets : GET (lista/singolo), POST, PUT, DELETE.
Program.cs	Configura EF Core + SQL Server, enum JSON come stringhe, health check su /health , migration automatica all'avvio.

Migration automatica all'avvio (`Database.Migrate()`): per un progetto con ambienti effimeri (destroy/apply frequenti), evita di dover rilanciare `dotnet ef database update` a mano ogni volta. In un servizio di produzione reale le migration andrebbero eseguite come step separato della pipeline CI/CD, non ad ogni avvio del container.

Autenticazione al database: in locale, `Authentication=Active Directory Default` (usa l'identità `az login` dello sviluppatore); in Container Apps, `Authentication=Active Directory Managed Identity;User Id=<client_id>` (usa la User-Assigned Identity `id-ticket-api-dev` della Container App). Stessa connection string "shape", provider diverso di credenziali a seconda del contesto — nessun cambio di codice tra i due.

7. Applicazione — ticket-web

Blazor Server (.NET 8, interattività globale server-side via SignalR) — UI CRUD ticket.

File	Responsabilità
Models/Ticket.cs	DTO — copia indipendente del modello di ticket-api (nessuna libreria condivisa: due soli servizi non giustificano quella complessità).
Services/ TicketApiClient.cs	Client HTTP tipizzato verso ticket-api: GET/POST/PUT/DELETE su <code>/api/tickets</code> . Configurato con <code>PropertyNameCaseInsensitive</code> e <code>JsonStringEnumConverter</code> per allinearsi al contratto JSON di ticket-api.
Components/Pages/ Tickets.razor	Tabella ticket + form inline di creazione/modifica/eliminazione.
Program.cs	Registra l' <code>HttpClient</code> tipizzato con <code>BaseAddress</code> da configurazione (<code>TicketApi:BaseUrl</code>).

Comunicazione con ticket-api: chiamata HTTP lato server (non dal browser) — nessun problema di CORS, dato che il browser parla solo con ticket-web via SignalR. In locale `BaseUrl` = `http://localhost:5286/` ; in Container Apps punta all'FQDN interno di ticket-api, calcolato automaticamente da Terraform come variabile d'ambiente della Container App.

8. Stato del progetto e prossimi passi

Step	Stato
Bootstrap state Terraform	Fatto
Ambiente shared (RG + ACR)	Fatto
Ambiente dev (RG, Log Analytics, Key Vault, SQL, Container Apps Environment)	Fatto
Applicazioni ticket-api + ticket-web	Fatto — testate end-to-end in locale
Build immagini + push su ACR	Fatto (<code>az acr build , tag dev</code>)
Deploy Container App reali (dev)	Fatto — verificato end-to-end anche pubblicamente (browser → ticket-web → ticket-api → SQL)
Provisioning Managed Identity come utente SQL (T-SQL)	Fatto (<code>sqlcmd , CREATE USER [id-ticket-api-dev] FROM EXTERNAL PROVIDER</code>)
Ambienti staging/prod	Da fare — solo backend.tf predisposto
OIDC GitHub Actions → Azure	Da fare
Pipeline Terraform (plan su PR, apply su merge, approvazioni staging→prod)	Da fare
Pipeline applicazione (build, push, deploy automatico)	Da fare
Tagging/governance finale	Da fare

URL pubblico attuale (dev): <https://ticket-web-dev.bravefield-c9746a9c.switzerlandnorth.azurecontainerapps.io/>

Note operative dal primo deploy

Il primo deploy delle Container App ha richiesto diverso debug — non per errori nel design dell'infrastruttura in sé, ma per problematiche pratiche tipiche del primo utilizzo di Container Apps + Managed Identity + ACR. Vale la pena registrarle:

- **Ordine identità/RBAC:** con System-Assigned Identity, il primo pull dell'immagine avviene durante la creazione stessa della Container App — troppo presto perché un ruolo RBAC assegnato subito dopo possa già essere efficace. Risolto passando a User-Assigned Identity con ruolo assegnato prima (vedi sezione 3.8).
- **Interruzioni di terraform apply :** interrompere con Ctrl+C un apply a metà può lasciare (a) lo state locale disallineato da quanto realmente creato su Azure (richiede `terraform import` per riconciliare) e (b) il lease sul blob dello state bloccato (richiede `az storage blob lease break`). Meglio lasciar sempre terminare un apply, anche se lento.

- **provisioningState: Failed non significa sempre "risorsa rotta"**: Container Apps a volte marca l'operazione di controllo come fallita (timeout interno) anche quando la risorsa converge correttamente poco dopo. L'ingress però non instrada traffico finché lo stato non torna `Succeeded` — un `az containerapp update --revision-suffix <nuovo>` forza un nuovo tentativo pulito.
- **Verifica sempre il contenuto reale delle immagini**: un errore banale (build lanciata dalla cartella sbagliata) ha fatto sì che l'immagine `ticket-web:dev` contenesse per errore il codice di `ticket-api`. I log applicativi (Log Analytics, tabella `ContainerAppConsoleLogs_CL`) sono stati decisivi per scoprirlo — uno stack trace di Entity Framework dentro quella che doveva essere un'app Blazor senza database è stato il segnale chiave.